# Verifying the HHL Algorithm in Isabelle/HOL

## Quantum Hoare Logic from the Spectral Theorem to Elementary Gates

Dustin Bryant

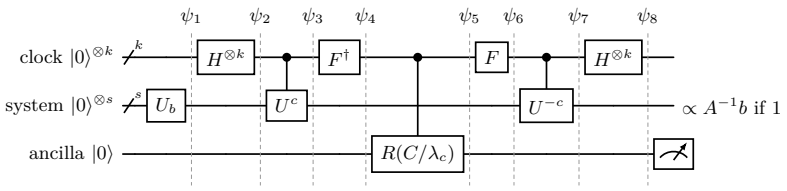
Independent, USA brydustin@gmail.com

**Abstract.** The algorithm of Harrow, Hassidim and Lloyd (HHL) prepares a quantum state proportional to the solution of a linear system $Ax = b$, for sparse and well-conditioned $A$ in time polylogarithmic in the dimension. It is the prototype of the quantum algorithms for linear algebra. Its correctness combines the spectral theorem, phase estimation, a controlled rotation, uncomputation, post-selection and amplitude amplification, and the error analysis in the paper's appendix is intricate. We verify HHL in Isabelle/HOL as programs of the quantum while-language of the AFP entry *QHLProver*, whose Hoare triples make probabilistic statements about measurements. For eigenvalues on the phase-estimation grid we prove total correctness: one run succeeds with probability $C^2\|A^{-1}b\|^2$, after which the state is exactly $|0\rangle\otimes A^{-1}b/\|A^{-1}b\|\otimes|1\rangle$, and the repeat-until-success loop terminates with probability one in that state, after $1/p$ rounds on average. For arbitrary spectra we prove Theorem 1 of the paper and its run-time bound with explicit constants, and fidelity triples for the corresponding programs. We then refine the programs until every gate is a one- or two-qubit unitary, including the preparation of $|b\rangle$, the controlled rotation and, for sparse $A$, a Trotter circuit for $e^{iAt}$ with explicit error. The formalisation corrects several statements of the HHL paper; in particular, the constant in its Lemma 3 is off by a factor of two. The development rests on a formalisation of Paulsen's lecture course on quantum computing and completely positive maps, and it builds without `sorry`, axioms or oracles.

**Keywords:** Isabelle/HOL · Quantum Hoare logic · HHL algorithm · Phase estimation · Hamiltonian simulation · Formal verification

## 1 Introduction

Solving a linear system $Ax = b$ is the workhorse of scientific computing. Harrow, Hassidim and Lloyd [13] observed that a quantum computer can prepare the state $|x\rangle \propto A^{-1}|b\rangle$ for a Hermitian $A \in \mathbb{C}^{N\times N}$ in time polynomial in $\log N$, in the condition number $\kappa$ and in $1/\varepsilon$, provided $A$ is sparse and $|b\rangle$ is easy to prepare, while classical algorithms take time polynomial in $N$. The algorithm, now called HHL, started the study of quantum algorithms for linear algebra and quantum machine learning. Its fine print (state preparation, sparse access, the




$\psi_1$ $|0\rangle \otimes b \otimes |0\rangle$
$\psi_2$ $u \otimes b \otimes |0\rangle$
$\psi_3$ $\sum_j |j\rangle \otimes N^{-1/2} U^j b \otimes |0\rangle$
$\psi_4$ $\sum_j |j\rangle \otimes \Pi_j b \otimes |0\rangle$
$\psi_5$ $\sum_j |j\rangle \otimes \Pi_j b \otimes R(C/\lambda_j)|0\rangle$
$\psi_6$ $\sum_j |j\rangle \otimes N^{-1/2}(U^j \otimes I)\, w$
$\psi_7$ $u \otimes w$
$\psi_8$ $|0\rangle \otimes w$

$$w = \sqrt{I - C^2 A^{-2}}\, b \otimes |0\rangle + C A^{-1} b \otimes |1\rangle$$

**Fig. 1.** The HHL circuit and the state after each of its eight gates, from $\psi_0 = |0\rangle|0\rangle|0\rangle$, for $N = 2^k$, $U = e^{iAt_0}$, $u = N^{-1/2}\sum_j |j\rangle$ and every eigenvalue of $A$ on the grid $\lambda_j = 2\pi j/(Nt_0)$ (taken as $2\pi(j-N)/(Nt_0)$ for $j \geq N/2$). Gate 3 applies $U^c$ when the clock holds $c$, gate 5 applies $R(C/\lambda_c)$ to the ancilla, where $R(c)|0\rangle = \sqrt{1-c^2}|0\rangle + c|1\rangle$, and $\Pi_j$ is the spectral projector of $A$ at $\lambda_j$. The table is the theorem hhl_steps (Section 4).

condition number, and the fact that the output is a quantum state rather than a list of numbers) has become part of how quantum speedups are assessed [1].

Figure 1 shows the circuit. Write $A = \sum_j \lambda_j |u_j\rangle\langle u_j|$ and $|b\rangle = \sum_j \beta_j |u_j\rangle$. Phase estimation with the unitaries $e^{iAt_0}$ writes an approximation $\tilde{\lambda}_j$ of each eigenvalue into a clock register, which gives the state $\sum_j \beta_j |\tilde{\lambda}_j\rangle|u_j\rangle$. A rotation of an ancilla qubit, controlled by the clock, produces the amplitudes $C/\tilde{\lambda}_j$. Uncomputing the clock and measuring the ancilla then leaves, on outcome 1, the state $\sum_j (\beta_j/\lambda_j)|u_j\rangle$, which is $A^{-1}|b\rangle$ up to normalisation. The success probability is $C^2\|A^{-1}b\|^2$, which is at least $1/\kappa^2$ for $C = \lambda_{\min}$ and unit $b$, so the measurement is repeated or amplified. Moreover, every step is approximate: the eigenvalues are not exactly representable in the clock, $e^{iAt_0}$ has to be simulated, and the condition number enters through the rotation angles. The appendix of the HHL paper bounds all of these errors.

HHL is a natural target for program verification. Its specification is probabilistic: it says what holds after a measurement and with which probability, and the full algorithm is a loop whose number of rounds is random. Quantum Hoare logic [31,11] was designed for such statements. A predicate is an operator $0 \leq P \leq I$, read as the observable "$P$ holds", and the triple $\{P\}\, S\, \{Q\}$ states that for every input the probability of $Q$ after running $S$ is at least the probability of $P$ before. HHL has been verified in this style in Coq: Zhou et al. [33] formalised the pen-and-paper proof of [34], and Wu, Deng and Xu [30] verified it in a logic with distribution formulas. Both treat the exact case, in which every eigenvalue satisfies $\lambda_j t_0/2\pi \in \{1, \ldots, N-1\}$. Both prove the partial-correctness triple $\{I\}\, HHL\, \{|x\rangle\langle x|\}$ for the repeat-until-success loop (whenever the loop terminates, the system holds $|x\rangle$), and both take $U_b$, the controlled rotation and the controlled evolution as given unitaries.

This paper reports a verification of HHL in Isabelle/HOL that goes beyond the exact case and below the given unitaries. Its theme is one kind of statement, a Hoare triple of the logic of Liu et al. [18,19] about a program, established at four levels (Fig. 2). The first is the mathematics behind the algorithm. The second is the algorithm with its error analysis. The third is the program, with initialisation, measurement and loop. The fourth is the circuit of elementary gates. Concretely, the contributions are as follows.

- *Exact HHL, total correctness* (Section 4). One run of the program succeeds with probability $C^2\|A^{-1}b\|^2$ and then holds exactly $|0\rangle \otimes A^{-1}b/\|A^{-1}b\| \otimes |1\rangle$. The repeat-until-success loop terminates with probability one in that state; it exits after exactly $m$ failures with probability $(1-p)^m p$ and needs $1/p$ rounds on average. The logic has no total-correctness rule for loops, so we derive one for loops whose body re-initialises the state.
- *The paper's regime* (Section 5). For eigenvalues anywhere in the spectral window, an inexact simulation oracle and the paper's filter functions, we prove the bounds of the paper's Theorem 1 and its run-time claim with explicit constants. We also prove fidelity triples $\{(1-e^2)\, I\}\ HHL\ \{|t\rangle\langle t|\}$ for the corresponding programs.
- *Elementary gates* (Section 6). The programs are refined, triple by triple, until every gate is a one- or two-qubit unitary on adjacent wires. This covers the Fourier transform; the controlled rotation, as a uniformly controlled rotation with Walsh–Hadamard angles; the preparation of the clock and of an arbitrary $|b\rangle$; and, for $s$-sparse $A$, a Trotter circuit for $e^{iAt}$ obtained from a graph colouring, with error $8(sLt)^2/r$ after $r(6s-2)$ queries.
- *Errata* (Section 7). Mechanising the appendix exposed several false statements in the HHL paper, among them the constant in Lemma 3, which must be $\pi^2$ rather than $\pi^2/2$. Each erratum reported here is backed by a formal counterexample or a corrected theorem.
- *The foundation* (Section 8). Everything rests on a formalisation of Vern Paulsen's lectures on quantum computing and completely positive maps [22]. Its 34 lectures are formalised in 50 theories and audited statement by statement. The spectral theorem and positivity theory of its first ten lectures carry the HHL development. Its later lectures develop the completely positive maps that give quantum programs their meaning [32].

All results are checked by Isabelle2025-2 with `quick_and_dirty = false`, so no `sorry` survives, and an audit with `Thm_Deps.all_oracles` over all theorems of the development finds no oracles. The sources, the generated documents and the audits are in the project repository.[1] Each listing below links to its source line ().

The rest of the paper is organised as follows. Section 2 recalls quantum Hoare logic and explains how HHL's specifications read as triples. Section 3 covers the functional calculus and phase estimation. Section 4 verifies exact HHL as a program, and Section 5 treats the regime of the paper's error analysis. Section 6

[1] `https://github.com/brydustin/Quantum_Computing`

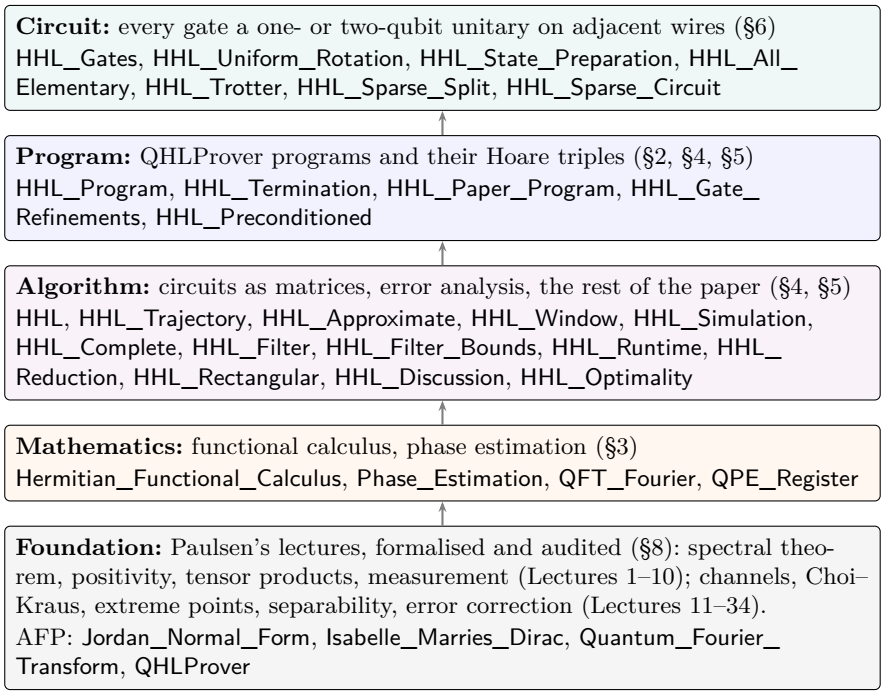



**Fig. 2.** The four levels of the HHL development and the theories at each level. Part VI of the development has 29 theories; the foundation has 50.

compiles the programs to elementary gates, and Section 7 collects the errata. Section 8 describes the foundation. Sections 9 and 10 discuss related work, the trusted base and the limitations.

## 2 Quantum Hoare Logic in Isabelle

### 2.1 States, Gates and Programs

We use the concrete representation of the Jordan-Normal-Form library [25], shared by Isabelle Marries Dirac [6,5], the quantum Fourier transform entry [20] and QHLProver [18,19]. A state is a vector v :: complex vec with a dimension hypothesis dim_vec v = n, an operator is a matrix with a carrier hypothesis A ∈ carrier_mat n n, ⊗ is the Kronecker product (tensor_vec for vectors), $|j\rangle$ is the unit vector unit_vec n j, $\langle v|w\rangle$ is antilinear in $v$, and $A^\dagger$ is the conjugate transpose. Dimensions are hypotheses rather than types, which is how the lecture notes and the physics literature treat them, and which lets the libraries share their definitions. The registers of HHL are ordered clock ⊗ system ⊗ ancilla, of dimensions $N = 2^k$, $n$ and 2.

QHLProver formalises Ying's quantum while-language [31,32]:

$$S ::= \textbf{skip} \mid U \mid S_1; S_2 \mid \textbf{measure}\ M\ [S_0, \ldots, S_{m-1}] \mid \textbf{while}\ M\ \textbf{do}\ S.$$

A measurement is a family $M_0, \ldots, M_{m-1}$ with $\sum_k M_k^\dagger M_k = I$, and a loop is controlled by a two-outcome measurement. Programs act on partial density operators ($\rho \geq 0$, $\operatorname{tr}\rho \leq 1$):

$$[\![U]\!]\rho = U\rho U^\dagger, \qquad [\![S_1; S_2]\!] = [\![S_2]\!] \circ [\![S_1]\!],$$
$$[\![\textbf{measure}\ M\ \bar{S}]\!]\rho = \textstyle\sum_k [\![S_k]\!](M_k \rho M_k^\dagger),$$
$$[\![\textbf{while}\ M\ \textbf{do}\ S]\!]\rho = \textstyle\sum_{m \geq 0} M_0 \left(([\![S]\!] \circ \mathcal{M}_1)^m \rho\right) M_0^\dagger, \quad \text{where } \mathcal{M}_1(\rho) = M_1 \rho M_1^\dagger.$$

The loop runs $S$ on outcome 1 and exits on outcome 0, and the $m$-th summand is the part of the input that exits after $m$ rounds. The meaning of a program is a completely positive, trace-non-increasing map, and the trace lost by a loop is the probability that it does not terminate.

### 2.2 Predicates and Triples

A quantum predicate is an operator $0 \leq P \leq I$. For a state $\rho$, $\operatorname{tr}(P\rho)$ is the expected value of the observable $P$; for a projector it is the probability that measuring $\{P, I - P\}$ yields $P$. A triple compares these probabilities before and after the program (Listing 1): $\models_t \{P\}\ S\ \{Q\}$ holds if $\operatorname{tr}(P\rho) \leq \operatorname{tr}(Q[\![S]\!]\rho)$ for every input $\rho$, and partial correctness adds the probability $\operatorname{tr}\rho - \operatorname{tr}[\![S]\!]\rho$ of not terminating to the right-hand side. These are D'Hondt and Panangaden's predicates [11] with Ying's notion of correctness [31].

```
definition is_quantum_predicate :: "complex mat ⇒ bool" where
  "is_quantum_predicate P ⟷ P ∈ carrier_mat d d ∧ positive P ∧ P ≤L
      1m d"
definition hoare_total_correct :: "complex mat ⇒ com ⇒ complex mat
    ⇒ bool" where
  "⊨t {P} S {Q} ⟷ (∀ρ ∈density_states. trace (P * ρ) ≤ trace (Q *
      denote S ρ))"
definition hoare_partial_correct :: "complex mat ⇒ com ⇒ complex
    mat ⇒ bool" where
  "⊨p {P} S {Q} ⟷ (∀ρ ∈density_states.
      trace (P * ρ) ≤ trace (Q * denote S ρ) + (trace ρ - trace
          (denote S ρ)))"
```

**Listing 1.** Quantum predicates and triples in QHLProver (inside the locale `state_sig`, where `d` is the dimension of the state space; syntax annotations omitted)

QHLProver also defines a proof system $\vdash_p$ for partial correctness and proves it sound and complete [18]:

$$\frac{}{\{U^\dagger P U\}\ U\ \{P\}} \qquad \frac{\{P\}\ S_1\ \{Q\} \quad \{Q\}\ S_2\ \{R\}}{\{P\}\ S_1; S_2\ \{R\}} \qquad \frac{P \leq P' \quad \{P'\}\ S\ \{Q'\} \quad Q' \leq Q}{\{P\}\ S\ \{Q\}}$$

$$\frac{\{P_k\}\ S_k\ \{Q\} \quad (k < m)}{\{\textstyle\sum_k M_k^\dagger P_k M_k\}\ \textbf{measure}\ M\ \bar{S}\ \{Q\}} \qquad \frac{\{Q\}\ S\ \{R\}}{\{R\}\ \textbf{while}\ M\ \textbf{do}\ S\ \{P\}}$$

**Table 1.** The four shapes of triple used for HHL, for a state $\rho$ with $\operatorname{tr}\rho = 1$.

| Triple | Reading |
|---|---|
| $\models_t \{p \cdot I\}\ S\ \{P_1\}$ | $S$ terminates with the ancilla in $\|1\rangle$ with probability $\geq p$ |
| $\models_t \{I\}\ S\ \{P_0 + \|t\rangle\langle t\|\}$ | whenever the ancilla reads 1 at the end, the state is exactly $\|t\rangle$ |
| $\models_t \{I\}\ S\ \{\|t\rangle\langle t\|\}$ | $S$ terminates with probability one, in the state $\|t\rangle$ |
| $\models_t \{q \cdot I\}\ S\ \{\|t\rangle\langle t\|\}$ | the output has fidelity at least $q$ with $\|t\rangle$ |

where $R = M_0^\dagger P M_0 + M_1^\dagger Q M_1$ in the loop rule, together with the rule for **skip**. The rules compute weakest preconditions backwards through a program. For loop-free programs, partial and total correctness coincide because such programs preserve the trace.

### 2.3 HHL's Specification as Triples

Every statement about HHL in this paper is a triple of one of four shapes (Table 1). Let $P_0$ and $P_1$ be the projectors onto ancilla value 0 and 1 (anc_proj), and let $|t\rangle$ be a unit vector with $P_1|t\rangle = |t\rangle$, typically $|t\rangle = |0\rangle \otimes A^{-1}b/\|A^{-1}b\| \otimes |1\rangle$. Since $\operatorname{tr}\rho = 1$ for a state, a precondition $q \cdot I$ asserts the number $q$ for every input. The second reading needs a short argument. Let $\sigma = [\![S]\!]\rho$ for a state $\rho$. Then $\operatorname{tr}\sigma \leq 1$, and $\operatorname{tr}((P_0 + |t\rangle\langle t|)\sigma) \geq 1$ forces $\operatorname{tr}((P_1 - |t\rangle\langle t|)\sigma) \leq 0$. Since $P_1 - |t\rangle\langle t| \geq 0$, this gives $P_1\sigma P_1 = \langle t|\sigma|t\rangle\, |t\rangle\langle t|$: whenever the ancilla reads 1, the state is exactly $|t\rangle$.

### 2.4 What We Add to the Logic

*Initialisation.* QHLProver has no statement that prepares a fixed state. We build one from a measurement in the computational basis followed by a permutation that moves the outcome $k$ to 0. Its rule says that after init_com the probability of $Q$ is $\langle 0|Q|0\rangle$, whatever the input.

```
definition init_com :: "nat ⇒ com" where
  "init_com K = Measure K (proj_basis K) (map (λk. Utrans (swap0 K
      k)) [0..<K])"
theorem hoare_init:
  assumes Q: "is_quantum_predicate Q" and d0: "0 < d"
  shows "⊢_p {Q $$ (0, 0) ·_m 1_m d} init_com d {Q}"
```

**Listing 2.** Initialisation: a measurement and $K$ swaps, and its proof rule

*Gate lists.* A circuit $U_1, \ldots, U_m$ becomes the program utrans_seq $[U_1, \ldots, U_m] = U_1; \ldots; U_m$. All theorems about HHL programs are proved for every list of unitaries whose product (circuit_mat) equals that of the eight gates of Fig. 1. A

refinement that replaces gates by circuits with the same product therefore inherits every triple, which is how Section 6 reaches elementary gates.

*Total correctness of loops.* The proof system covers partial correctness only. For loops whose body re-initialises the state, we derive total correctness from the semantics (Section 4.3).

*Fidelity triples.* For a program that initialises, runs a circuit and measures, the probability of $|t\rangle\langle t|$ is the overlap $|\langle t|\psi\rangle|^2$ with the circuit's output $\psi$. Section 5.4 uses this to turn the error bounds of the algorithm into triples.

## 3 The Mathematics: Functional Calculus and Phase Estimation

### 3.1 A Functional Calculus for Hermitian Matrices

HHL applies three functions to $A$: the evolution $e^{iAt}$, the inverse $A^{-1}$, and $x \mapsto \sqrt{1 - C^2/x^2}$, which appears in the output. We define $g(A)$ for Hermitian $A$ and any $g : \mathbb{R} \to \mathbb{C}$ as the matrix that acts on every eigenvector by the value of $g$ at its eigenvalue (Listing 3).

**Theorem 1 (functional calculus).** *Let $A \in \mathbb{C}^{n\times n}$ be Hermitian. There is exactly one matrix $g(A)$ with $g(A)v = g(\lambda)v$ for every eigenvector $v$ of $A$ with eigenvalue $\lambda$, and $g(A) = U\,\mathrm{diag}(g(d_1), \ldots, g(d_n))U^\dagger$ for every spectral decomposition $U^\dagger AU = \mathrm{diag}(d_1, \ldots, d_n)$.*

Existence comes from the spectral theorem of Paulsen's Lecture 4 (hermitian_spectral_decomp), and uniqueness from the fact that the eigenvectors span. Multiplicativity $g(A)h(A) = (gh)(A)$, $g(A)^\dagger = \bar{g}(A)$, unitarity when $|g| = 1$ on the spectrum, and positivity when $g \geq 0$ on it are all checked on eigenvectors. The instances are evolution $A\ t = e^{itA}$, with its group law, and herm_inv $A = A^{-1}$. Since $1/0 = 0$ in HOL, herm_inv is the pseudo-inverse when 0 is an eigenvalue (Section 4.4). The device behind the whole development is the following lemma: two matrices that agree on every eigenvector of a Hermitian $A$ are equal (eigvec_mat_eql). It lets every statement about all right-hand sides $b$ be proved for an eigenvector, without sums over an eigenbasis.

```
definition fun_on_eigen :: "complex mat ⇒ (real ⇒ complex) ⇒
    complex mat ⇒ bool" where
  "fun_on_eigen A g B ⟷ (∀v l. dim_vec v = dim_row A ⟶
     A *_v v = complex_of_real l ·_v v ⟶ B *_v v = g l ·_v v)"
definition mat_fun :: "(real ⇒ complex) ⇒ complex mat ⇒ complex
    mat" where
  "mat_fun g A = (SOME B. B ∈ carrier_mat (dim_row A) (dim_row A)
      ∧ fun_on_eigen A g B)"
theorem mat_fun_eigen:
  assumes A: "A ∈ carrier_mat n n" "hermitian A"
      and v: "dim_vec v = n" and ev: "A *_v v = complex_of_real l ·_v
          v"
  shows "mat_fun g A *_v v = g l ·_v v"
theorem mat_fun_decomp:
  assumes A: "A ∈ carrier_mat n n" "hermitian A" and sd:
      "spectral_decomp A U D"
  shows "mat_fun g A = U * diag_cmat (map g D) * (U†)"
```

**Listing 3.** The functional calculus

### 3.2 Phase Estimation

Let $F_N$ be the Fourier matrix, $(F_N)_{jk} = \omega^{jk}/\sqrt{N}$ with $\omega = e^{2\pi i/N}$, and let $\Lambda(U) = \sum_c |c\rangle\langle c| \otimes U^c$ be the controlled powers of $U$ (ctrl_pow). Phase estimation is $(F_N^\dagger \otimes I)\Lambda(U)$ applied to the uniform clock state $u = N^{-1/2}\sum_c |c\rangle$ (Fig. 1, gates 2–4).

**Theorem 2 (phase estimation).** *If $Uv = e^{2\pi i\varphi}v$, then $(F_N^\dagger \otimes I)\Lambda(U)(u \otimes v) = (\sum_j \alpha_j |j\rangle) \otimes v$ with $\alpha_j = \frac{1}{N}\sum_{k<N} e^{2\pi i k(\varphi - j/N)}$. If $\varphi = j_0/N$, then $\alpha = |j_0\rangle$. If $|\varphi - j/N| \le 1/(2N)$, then $|\alpha_j|^2 \ge 4/\pi^2$.*

The first part is phase kickback: $\Lambda(U)(|c\rangle \otimes v) = e^{2\pi i c\varphi}|c\rangle \otimes v$. The bound follows from the geometric sum, $|e^{ix} - 1| = 2|\sin(x/2)|$ and Jordan's inequality $\sin x \ge 2x/\pi$ on $[0, \pi/2]$. Two bridges connect the theorem to circuits. The quantum Fourier transform circuit of the AFP [20] is the Fourier matrix (ordered_QFT_fourier, by induction over its Kronecker structure), and $k$ Hadamard gates prepare the uniform state from $|0\cdots0\rangle$ (hadamard_layer_zero, from Isabelle Marries Dirac).

```
theorem phase_estimation:
  assumes U: "U ∈ carrier_mat m m" and u: "dim_vec u = m"
      and ev: "U *_v u = cis (2 * pi * φ) ·_v u"
  shows "(((fourier N)†) ⊗ 1_m m) *_v (ctrl_pow N m U *_v tensor_vec
      (uniform_state N) u) =
         tensor_vec (Matrix.vec N (pe_amp N φ)) u"
theorem phase_estimation_bound:
  assumes N: "0 < N" and close: "|φ - real j / real N| ≤ 1 / (2 *
      real N)"
  shows "4 / pi^2 ≤ (cmod (pe_amp N φ j))^2"
theorem ordered_QFT_fourier: "ordered_QFT n = fourier (2 ^ n)"
theorem hadamard_layer_zero:
  assumes n: "1 ≤ n"
  shows "(Hadamard ⊗^n) *_v unit_vec (2 ^ n) 0 = uniform_state (2 ^
      n)"
```

**Listing 4.** Phase estimation, the $4/\pi^2$ bound, and the bridges to circuits

## 4 Exact HHL as a Verified Program

In this section every eigenvalue of $A$ is a grid value: $\lambda_k = 2\pi k/(Nt_0)$ for $k < N/2$ and $2\pi(k-N)/(Nt_0)$ otherwise (reg_eig), so that $e^{it_0\lambda_k} = \omega^k$ and phase estimation is exact. This is the setting of the Coq proofs [33,30]; Section 5 removes it.

### 4.1 The Circuit

The circuit is $W^\dagger RW$, with phase estimation $W = (F_N^\dagger \otimes I)\Lambda(e^{iAt_0} \otimes I)$ and the controlled rotation $R = \sum_k |k\rangle\langle k| \otimes I \otimes R(C/\lambda_k)$. Here $R(c) = \left(\begin{smallmatrix} \sqrt{1-c^2} & -c \\ c & \sqrt{1-c^2} \end{smallmatrix}\right)$ is unitary for $|c| \le 1$.

**Theorem 3 (exact HHL).** *Let $A \in \mathbb{C}^{n\times n}$ be Hermitian with every eigenvalue a nonzero grid value $\lambda_k$, $0 < k < N$. Then for every $b \in \mathbb{C}^n$ and every real $C$,*

$$W^\dagger RW\,(u \otimes b \otimes |0\rangle) = u \otimes \left(\sqrt{I - C^2A^{-2}}\,b \otimes |0\rangle + CA^{-1}b \otimes |1\rangle\right).$$

*Hence $x = A^{-1}b$ solves $Ax = b$, the ancilla reads 1 with probability $C^2\|x\|^2$, and if $C \le |\lambda| \le \Lambda$ on the spectrum, this probability is at least $(C/\Lambda)^2\|b\|^2$, which is $1/\kappa^2$ for unit $b$ and $C = \lambda_{\min}$.*

```
theorem hhl:
  assumes A: "A ∈ carrier_mat n n" "hermitian A" and t0: "0 < t0"
      and N: "0 < N"
      and spec: "⋀l. l ∈ herm_spectrum A ⟹ ∃k. 0 < k ∧ k < N ∧ l =
          reg_eig t0 N k"
      and b: "dim_vec b = n"
  shows "hhl_circuit A t0 N C *v tensor_vec (uniform_state N)
      (tensor_vec b (unit_vec 2 0)) =
         tensor_vec (uniform_state N)
           (tensor_vec (mat_fun (λx. complex_of_real (sqrt (1 - (C
               / x)^2))) A *v b) (unit_vec 2 0) +
            tensor_vec ((complex_of_real C ·m hermitian_inv A) *v b)
                (unit_vec 2 1))"
```

**Listing 5.** Exact HHL as a matrix identity, for every right-hand side

The proof first takes $b$ to be an eigenvector with eigenvalue $\lambda_k$ (hhl_eigen). Phase estimation sends $u \otimes b$ to $|k\rangle \otimes b$ by Theorem 2, the rotation acts on the single branch $k$, and $W^\dagger$ undoes $W$. Both sides of the identity are linear in $b$ and of the form $Mb$ for matrices $M$ built with lift_mat, so eigvec_mat_eql extends the identity to all $b$. The success bound applies the functional calculus to $g(x) = 1/x^2 - 1/\Lambda^2$, which is non-negative on the spectrum, so $g(A)$ is positive (hhl_success_bound).

### 4.2 The Program and its Trajectory

As a program, HHL starts from $|0\rangle|0\rangle|0\rangle$. It prepares $|b\rangle = U_b|0\rangle$ and the uniform clock with Hadamard gates, so the circuit has eight gates (hhl_gates, Fig. 1). The table of Fig. 1 is the theorem hhl_steps, which gives the state after each gate for every $b$. Phase estimation leaves $\sum_j |j\rangle \otimes \Pi_j b \otimes |0\rangle$, where $\Pi_j = g_j(A)$ for the indicator $g_j$ of $\{\lambda_j\}$. With the unitary rule and the rule of consequence, each line becomes a triple $\{|\psi_i\rangle\langle\psi_i|\}\, U_i\, \{|\psi_{i+1}\rangle\langle\psi_{i+1}|\}$ (hhl_step_triples). The program initialises all registers, runs the gates and measures the ancilla (Listing 6).

```
definition hhl_prog :: "nat ⇒ complex mat list ⇒ com" where
  "hhl_prog K Us = init_com K ;; utrans_seq Us ;; Measure 2
      (anc_proj K) [SKIP, SKIP]"
definition hhl_rus_prog :: "nat ⇒ complex mat list ⇒ com" where
  "hhl_rus_prog K Us = init_com K ;; utrans_seq Us ;;
                       While (λj. anc_proj K (1 - j)) (init_com K
                           ;; utrans_seq Us)"
definition hhl_program :: "complex mat ⇒ complex mat ⇒ real ⇒ nat
    ⇒ real ⇒ com" where
  "hhl_program A Ub t0 k C = hhl_prog (2 ^ k * (dim_row A * 2))
      (hhl_gates A Ub t0 k C)"
```

**Listing 6.** The HHL programs: one run, and repeat until the ancilla reads 1

**Theorem 4 (exact HHL, one run).** *Let $A$ be as in Theorem 3 with $N = 2^k$, let $U_b$ be a unitary with $U_b|0\rangle = b$, and let $|C/\lambda_j| \le 1$ for all $j < N$. Let*

$p = C^2\|A^{-1}b\|^2$ *and* $|t\rangle = |0\rangle \otimes A^{-1}b/\|A^{-1}b\| \otimes |1\rangle$. *Then*

$$\models_t \{p \cdot I\}\ \mathit{hhl_program}\ \{P_1\} \qquad \textit{and} \qquad \models_t \{I\}\ \mathit{hhl_program}\ \{P_0 + |t\rangle\langle t|\}.$$

By Table 1, a single run succeeds with probability at least $p$, and on success the state is exactly $|t\rangle$. The hypothesis on $C$ makes the rotation gate unitary, as the program semantics requires. Listing 7 shows the formal statement, including the context that fixes the hypotheses.

```
context
  fixes A Ub :: "complex mat" and b :: "complex vec" and n k ::
      nat and t0 C :: real
  assumes A: "A ∈ carrier_mat n n" "hermitian A" and n0: "0 < n"
      and k: "1 ≤ k" and t0: "0 < t0"
    and spec: "⋀l. l ∈ herm_spectrum A ⟹ ∃j. 0 < j ∧ j < 2 ^ k ∧ l
       = reg_eig t0 (2 ^ k) j"
    and Ub: "Ub ∈ carrier_mat n n" "unitary Ub" "Ub *v unit_vec n
       0 = b"
    and Cb: "⋀j. j < 2 ^ k ⟹ |C / reg_eig t0 (2 ^ k) j| ≤ 1"
begin
theorem hhl_success_total:
  "state_sig.hoare_total_correct [2 ^ k, n, 2]
     (complex_of_real (C^2 * ‖herm_inv A *v b‖^2) ·m 1m (2 ^ k * (n
         * 2)))
     (hhl_program A Ub t0 k C) (anc_proj (2 ^ k * (n * 2)) 1)"
theorem hhl_correct_total:
  "state_sig.hoare_total_correct [2 ^ k, n, 2] (1m (2 ^ k * (n *
      2))) (hhl_program A Ub t0 k C)
     (anc_proj (2 ^ k * (n * 2)) 0 + outer_prod (hhl_target A k b)
         (hhl_target A k b))"
```

**Listing 7.** Theorem 4 (state_sig.hoare_total_correct dims is $\models_t$ on the registers dims)

The partial-correctness versions are derived with the proof rules (hhl_success, hhl_correct). The postcondition is pulled back through the measurement with the measurement rule, through the gates with the unitary rule, which along the trajectory turns it into a predicate whose value on $|000\rangle$ is $p$ (respectively 1), and through the initialisation with hoare_init. Total correctness follows because programs without loops preserve the trace (denote_trace_eq). All of this is proved for any gate list with the circuit of the eight gates (hhl_prog_success, hhl_prog_correct).

### 4.3 Repeat Until Success

A failed run (ancilla 0) is discarded and the program is run again. This is the loop hhl_rus_prog of Listing 6, which exits when the ancilla reads 1. Partial correctness, $\models_p \{I\}\ \mathit{hhl_rus}\ \{|t\rangle\langle t|\}$ (hhl_rus_correct), follows from the loop rule. Its invariant is $P_0 + |t\rangle\langle t|$, the postcondition of the second triple of Theorem 4. It says nothing about termination, and QHLProver has no rule for total correctness of loops. We therefore work in the semantics. Because the

body begins with init_com, it maps every input $\rho$ to $(\operatorname{tr}\rho)\,\sigma$ for the fixed state $\sigma = [\![init; U_1; \ldots; U_8]\!](|0\rangle\langle 0|)$. The $m$-th summand of the loop semantics is then $(1-p)^m$ times the exit of one round, and the loop is a geometric series.

**Lemma 1 (loops with a resetting body).** *Let $M$ be a two-outcome measurement and $S$ a body with $[\![S]\!]\rho = (\operatorname{tr}\rho)\,\sigma$ for every partial density operator $\rho$, where $\operatorname{tr}\sigma = 1$, $\operatorname{tr}(M_1\sigma M_1^\dagger) = 1-p$ and $0 < p \le 1$. Let $Q$ be a predicate and $v = \operatorname{tr}(QM_0\sigma M_0^\dagger)$. Then $\operatorname{tr}\big(Q\,[\![\textbf{while } M \textbf{ do } S]\!](c\sigma)\big) \ge cv/p$ for $0 \le c \le 1$.*

```
lemma while_reset_total:
  assumes M: "measurement d 2 M" and Sc: "well_com Sc"
    and σ: "σ ∈ carrier_mat d d" "partial_density_operator σ"
        "trace σ = 1"
    and reset: "⋀ρ. ρ ∈ carrier_mat d d ⟹ partial_density_operator
        ρ ⟹ denote Sc ρ = trace ρ ·_m σ"
    and p: "trace (M 1 * σ * adjoint (M 1)) = complex_of_real (1 -
        p)" "0 < p" "p ≤ 1"
    and Q: "is_quantum_predicate Q" and v: "trace (Q * (M 0 * σ *
        adjoint (M 0))) = complex_of_real v"
    and c: "0 ≤ c" "c ≤ 1"
  shows "complex_of_real (c * v / p) ≤ trace (Q * denote (While M
      Sc) (complex_of_real c ·_m σ))"
```

**Listing 8.** Lemma 1

For HHL, $M_0 = P_1$ and the exit of one round is $p\,|t\rangle\langle t|$, so $Q = |t\rangle\langle t|$ gives $v = p$. The probability of $|t\rangle$ at the end is then at least the trace of the input, which is total correctness.

**Theorem 5 (exact HHL, repeat until success).** *Under the hypotheses of Theorem 4 and $C \neq 0$:*

1. *$\models_t \{I\}$ hhl_rus $\{|t\rangle\langle t|\}$, i.e. the loop terminates with probability one in $|t\rangle$;*
2. *after exactly $m$ failed rounds the loop exits with probability $(1-p)^m p$, in the state $|t\rangle$, and is still running with probability $(1-p)^m$;*
3. *the expected number of rounds is $\sum_m (m+1)p(1-p)^m = 1/p$.*

```
theorem hhl_rus_total:
  assumes "C ≠ 0"
  shows "state_sig.hoare_total_correct [2 ^ k, n, 2] (1_m (2 ^ k *
      (n * 2))) (hhl_rus A Ub t0 k C)
     (outer_prod (hhl_target A k b) (hhl_target A k b))"
theorem hhl_rus_expected_rounds:
  assumes C0: "C ≠ 0"
  defines "p ≡ C² * ‖herm_inv A *_v b‖²"
  shows "(λm. real (Suc m) * (p * (1 - p) ^ m)) sums (1 / p)" and
      "(λm. (1 - p) ^ m) sums (1 / p)"
```

**Listing 9.** Theorem 5(1) and (3), in the context of Listing 7

Part (2) is hhl_rus_prog_rounds, which computes the $m$-th iterate of the loop semantics in closed form. With $C = \lambda_{\min}$ and unit $b$, the expected number of rounds is at most $\kappa^2$; amplitude amplification reduces this to $O(\kappa)$ (Section 5).

### 4.4 Other Systems

For an invertible non-Hermitian $A$, the program runs on the Hermitian dilation $\tilde{A} = \left(\begin{smallmatrix} 0 & A \\ A^\dagger & 0 \end{smallmatrix}\right)$, and $\tilde{A}^{-1}$ maps $b \oplus 0$ to $0 \oplus x$ (herm_embed_inverse). A preconditioned system $BAx = Bb$ is run in the same way. On success the system register holds $0 \oplus x/\|x\|$ for the solution of the original $Ax = b$, and the success probability is at least $(C/s)^2$ when $\|BAv\| \le s\|v\|$ (hhl_elem_precond_correct, hhl_elem_precond_bound). For a rectangular $A \in \mathbb{C}^{M\times N}$, the pseudo-inverse of the dilation maps $b \oplus 0$ to $0 \oplus x$ with $A^\dagger Ax = A^\dagger b$ and $x$ in the range of $A^\dagger$. So $x$ is the minimum-norm solution when $Ax = b$ is solvable, and a least-squares solution otherwise (pinv_solution, normal_eq_least_squares). The proof uses $Jg(H)J = g(-H)$ for the parity $J$, and it is here that $1/0 = 0$, which makes herm_inv the pseudo-inverse, pays off.

## 5 Beyond Exact Phase Estimation: the Paper's Error Analysis

In general the eigenvalues of $A$ lie anywhere in a spectral window, $e^{iAt_0}$ is available only approximately, and the success probability must be amplified. This section follows the paper's analysis [14, App. A] and ends with programs whose triples carry the resulting error bounds.

### 5.1 Inexact Phase Estimation and the Sine Window

If $\lambda$ is not a grid value, phase estimation leaves the clock in a superposition $\sum_j \alpha_j|j\rangle$, the rotation acts on branch $j$ with angle $C/\lambda_j$, and uncomputation is inexact. We write states as register sums $\sum_j |j\rangle \otimes f_j$ (reg_sum). Block operators act branch by branch and branches are orthogonal, which gives an exact identity for an eigenvector $v$ (hhl_approx_error):

$$\left\| \text{out} - u \otimes v \otimes R(c)|0\rangle \right\|^2 = \|v\|^2 \sum_j |\alpha_j|^2 \, \|R(c_j)|0\rangle - R(c)|0\rangle\|^2,$$

where $c_j = C/\lambda_j$ and $c = C/\lambda$. For grid values below $C$ the coefficient $c_j$ exceeds 1 and $R(c_j)$ is not unitary, so the physical circuit clamps it to $[-1, 1]$ (hhl_circuit_clamped). The error vectors of distinct eigenvectors are orthogonal, so the bound for eigenvectors extends to every $b$ with no loss in the dimension. With the uniform clock state, the amplitudes decay only like $|\alpha_j| \le 1/(2N|\delta_j|)$, where $\delta_j = \varphi - j/N$, and the squared error is $O(1/N)$ (hhl_clamped_bound). Precision $\varepsilon$ would then need $N \sim 1/\varepsilon^2$. The paper prepares the clock in the sine window $\psi_\tau = \sqrt{2/N}\,\sin(\pi(\tau + \frac{1}{2})/N)$ instead, whose amplitudes decay like $\delta^{-2}$. From a closed form of the window sum we prove $|\alpha_j|^2 \le \pi^2/(2N^4\delta_j^4)$ for $1/N \le |\delta_j| \le \frac{1}{2}$ (win_amp_tail). This gives the following theorem.

**Theorem 6 (window register).** *Let $A$ be Hermitian, let its spectrum lie in the interval $[\lambda_{\min}, \pi/(2t_0)]$, and let $0 < C$, $0 < \theta < 1$ and $2C/\theta \le \lambda_{\min}$. Then for every $b$, the clamped circuit with the window register satisfies*

$$\|\mathrm{out} - \psi \otimes w\|^2 \le \Big(\frac{K_1}{N^2} + \frac{K_2}{N^3}\Big)\|b\|^2,$$

*where $w$ is the exact output of Theorem 3, $K_1 = 8\pi^2(2+\pi^2)\theta^4/(t_0^2C^2(1-\theta^2))$ and $K_2 = 512\pi^6/(t_0^4\lambda_{\min}^4)$. Hence $N \ge Q/\varepsilon$ gives error at most $\varepsilon\|b\|$, for an explicit $Q$.*

```
theorem hhl_window_bound:
  assumes A: "A ∈ carrier_mat n n" "hermitian A" and t0: "0 < t0"
      and N: "2 ≤ N"
      and C: "0 < C" and th: "0 < θ" "θ < 1" and lmin: "2 * C / θ
          ≤ lmin"
      and spec: "⋀l. l ∈ herm_spectrum A ⟹ lmin ≤ l ∧ t0 * l ≤ pi
          / 2"
      and b: "dim_vec b = n"
  shows "Re ⟨Δ|Δ⟩ ≤ (8 * pi² * (2 + pi²) * θ ^ 4 / ((real N)² * t0² *
      C² * (1 - θ²))
                      + 512 * pi ^ 6 / ((real N) ^ 3 * t0 ^ 4 *
                         lmin ^ 4)) * Re ⟨b|b⟩"
  (* Δ = hhl_circuit_clamped A t0 N C *v (win_state N ⊗ b ⊗ |0⟩) -
      win_state N ⊗ hhl_out A C b *)
```

**Listing 10.** Theorem 6 (abridged: $\Delta$ is written out in the source)

On branches close to the eigenvalue the distance is $O(\delta_j^2)$, so each term is at most $K\,w(N\varphi - j)/N^2$ for a weight $w$ with $\sum_j w(N\varphi - j) \le 4 + 2\pi^2$. Clamped and negative branches have $|\delta_j| \ge \varphi/2$ and contribute $O(1/N^3)$ in total.

### 5.2 Filter Functions and Theorem 1 of the Paper

The paper's appendix replaces the ancilla by a three-level flag register. The state $|h(\lambda)\rangle = \sqrt{1-f^2-g^2}\,|nothing\rangle + f|well\rangle + g|ill\rangle$ is built from two filter functions (Fig. 3). The well-conditioned part $f = 1/(2\kappa\lambda)$ for $\lambda \ge 1/\kappa$ inverts the matrix, and $g$ flags the ill-conditioned eigenvalues below $1/(2\kappa)$. Lemma 2 of the paper states that $\lambda \mapsto |h(\lambda)\rangle$ is $\frac{\pi}{2}\kappa$-Lipschitz, and we prove it with that constant (hflag_lipschitz). Lemma 3 is a relative version:

$$|f(\lambda) - f(\tilde\lambda)|^2 + |g(\lambda) - g(\tilde\lambda)|^2 \;\le\; c\,\kappa^2(\lambda - \tilde\lambda)^2\big(f(\lambda)^2 + g(\lambda)^2\big),$$

stated with $c = \pi^2/2$. Its proof uses $f^2 + g^2 = \frac{1}{2}$ on the middle region $[\frac{1}{2\kappa}, \frac{1}{\kappa})$, but there $f^2 + g^2 = \frac{1}{4}$ (Fig. 3). The lemma is false with $c = \pi^2/2$ and true with $\pi^2$, and $\pi^2$ cannot be improved (Listing 11).

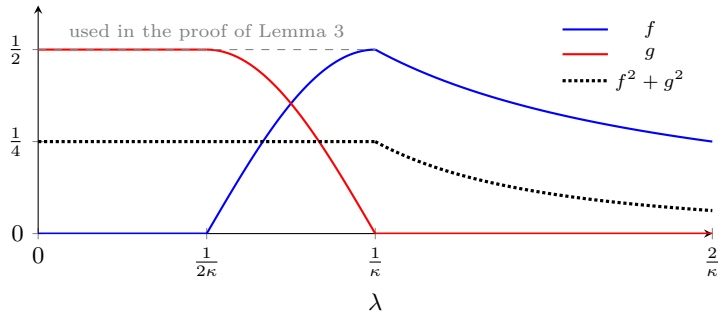



**Fig. 3.** The filter functions of the HHL paper for $\kappa = 2$. On $[\frac{1}{2\kappa}, \frac{1}{\kappa})$ the point $(f, g)$ moves on a circle of radius $\frac{1}{2}$ with angular speed $\pi\kappa$. So $|h(\lambda)\rangle$ moves with speed exactly $\frac{\pi}{2}\kappa$ there (Lemma 2), $f^2 + g^2 = \frac{1}{4}$ rather than the $\frac{1}{2}$ used in the paper's proof of Lemma 3, and the constant of Lemma 3 is $\pi^2$.

```
definition filt_f :: "real ⇒ real ⇒ real" where
  "filt_f κ x = (if 1 / κ ≤ x then 1 / (2 * κ * x)
                 else if 1 / (2 * κ) ≤ x then sin (pi / 2 * (2 *
                     κ * x - 1)) / 2 else 0)"
definition filt_g :: "real ⇒ real ⇒ real" where
  "filt_g κ x = (if 1 / κ ≤ x then 0
                 else if 1 / (2 * κ) ≤ x then cos (pi / 2 * (2 *
                     κ * x - 1)) / 2 else 1 / 2)"
theorem lemma3:
  "(filt_f κ a - filt_f κ b)^2 + (filt_g κ a - filt_g κ b)^2
     ≤ pi^2 * κ^2 * (a - b)^2 * ((filt_f κ a)^2 + (filt_g κ a)^2)"
theorem lemma3_paper_constant:
  "pi^2 / 2 * κ^2 * (1 / (2 * κ) - 1 / κ)^2 * ((filt_f κ (1 / (2 *
      κ)))^2 + (filt_g κ (1 / (2 * κ))))^2)
   < (filt_f κ (1 / (2 * κ)) - filt_f κ (1 / κ))^2 + (filt_g κ (1 /
       (2 * κ)) - filt_g κ (1 / κ))^2"
theorem lemma3_sharp:
  assumes c: "c < pi^2"
  shows "∃a b. c * κ^2 * (a - b)^2 * ((filt_f κ a)^2 + (filt_g κ a)^2)
                < (filt_f κ a - filt_f κ b)^2 + (filt_g κ a -
                    filt_g κ b)^2"
```

**Listing 11.** The filter functions, Lemma 3 with the corrected constant, the counterexample to the paper's constant, and sharpness (for fixed $\kappa > 0$)

The counterexample is $\lambda = \frac{1}{2\kappa}$, $\tilde{\lambda} = \frac{1}{\kappa}$, where the two sides are $\frac{1}{2}$ and $\pi^2/32$. Sharpness takes both points in the middle region, where the left side is $\sin^2(\pi\kappa(\lambda - \tilde{\lambda})/2)$, and uses $\sin w \geq w - w^3/6$. The paper uses Lemma 3 for the two error terms of the post-selected output, (A13)–(A17), and states Theorem 1 only up to $O(\kappa/t_0)$, so the corrected constant does not change that statement. With the corrected lemma, the window register and the wrapped distances of the grid values, all three parts of Theorem 1 of the paper hold for every $b$

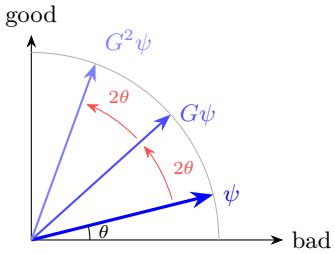



**Fig. 4.** Amplitude amplification. The Grover iterate $G = (2|\psi\rangle\langle\psi| - I)(I - 2P)$ rotates the plane of the good component $P\psi$ and the bad component by $2\theta$, where $\sin\theta = \|P\psi\|$. After $k$ rounds the good amplitude is $\sin((2k+1)\theta)$: $PG^k\psi = \frac{\sin((2k+1)\theta)}{\sin\theta} P\psi$ (amplitude_amplification).

with explicit constants. For $t = Nt_0$ the total evolution time, $\|(\tilde{U} - U)b\| \le 3\pi^2\sqrt{4+2\pi^2}\,\kappa\|b\|/t$, and the states post-selected on $\{well, ill\}$, or on *well* when $b$ lies in the well-conditioned subspace, are within $12\pi^2\sqrt{4+2\pi^2}\,\kappa/t$ of the ideal ones (flag_bound, flag_post_normalized, flag_well_normalized).

### 5.3 Oracles, Amplification and Run Time

The circuit uses $e^{iAt_0}$ only through controlled powers. We model it in a small language of fixed gates, controlled calls to an oracle $V$, and sequencing (qcirc), with a matrix semantics (sem) and a call count (calls). HHL makes $2(N-1)$ calls, and replacing the oracle by any unitary $V$ with $\|Vv - e^{iAt_0}v\| \le \eta\|v\|$ moves the output by at most $2(N-1)\eta$, because each call adds at most $\eta$ (hhl_sim_robust). Post-selection is stable: if the output is within $\delta$ of the exact one, the post-selected state is within $2\delta/(C\|A^{-1}b\|)$ of $\psi \otimes A^{-1}b/\|A^{-1}b\| \otimes |1\rangle$ (hhl_readout). Amplitude amplification is proved in the plane spanned by the good and bad components (Fig. 4). As a circuit, each round uses the HHL circuit and its inverse once.

The paper states its run time in a normalisation where the spectrum lies in $[1/\kappa, 1]$. In that normalisation the components combine as follows.

**Theorem 7 (run time).** *Let the spectrum of $A$ lie in $[1/\kappa, 1]$, let $t_0 \le \pi/2$, $\varepsilon\kappa \le 1$ and $Nt_0 \ge 12\pi^2\sqrt{4+2\pi^2}\,\kappa/\varepsilon$, and let the flag gates prepare $|h(\tilde{\lambda}_k)\rangle$. Then the post-selected well-conditioned output is within $\varepsilon$ of $A^{-1}b/\|A^{-1}b\|$, its amplitude is at least $1/(4\kappa)$, and at most $\pi\kappa$ rounds of amplitude amplification, each with $2(N-1)$ oracle calls, succeed with probability at least $1 - \|y\|^2$, where $y$ is the well-flagged part of one run.*

```
theorem hhl_paper_runtime:
  assumes ... "⋀l. l ∈ herm_spectrum A ⟹ 1 / κ ≤ l ∧ l ≤ 1"
      and ... "0 < ε" "ε * κ ≤ 1" and "12 * pi² * sqrt (4 + 2 *
          pi²) * κ / ε ≤ real N * t0"
  defines "out ≡ sem N (n * 3) (evolution A t0 ⊗ 1_m 3)
      (flag_full_prog N n G Prep) *_v unit_vec (N * (n * 3)) 0"
  defines "y ≡ pflag 3 (λc. c = 1) out" and "x ≡ herm_inv A *_v b"
  shows "‖complex_of_real (1 / ‖y‖) ·_v y - tensor_vec (win_state
      N)
            (tensor_vec (complex_of_real (1 / ‖x‖) ·_v x)
                (unit_vec 3 1))‖ ≤ ε"
    and "1 / (4 * κ) ≤ ‖y‖"
    and "‖y‖ < 1 ⟹ ∃k. 1 - ‖y‖² ≤ ‖pflag 3 (λc. c = 1) (sem
        ... (aa_progP ... k) ...)‖²
           ∧ calls (aa_progP (N * (n * 3)) 3 (λc. c = 1)
               (flag_full_prog N n G Prep) k) = (2 * k + 1) * (2 *
               (N - 1))
           ∧ real k ≤ pi * κ"
```

**Listing 12.** Theorem 7 (hypotheses abridged; `out` is the output of the flag circuit and `y` its well-flagged part)

Each round runs the circuit and its inverse, so the total simulated time is $(2k+1)\cdot 2(N-1)t_0 = O(\kappa^2/\varepsilon)$, and the state preparation is used $O(\kappa)$ times. The number of rounds depends on the unknown success amplitude. Following the paper, we use the averaging lemma of Brassard, Høyer, Mosca and Tapp [7]. For $k$ uniform below $M \geq 1/\sin 2\theta$, the mean of $\sin^2((2k+1)\theta)$ is at least $\frac{1}{4}$ (bhmt_quarter). Stages with $k$ uniform below $1, 2, 4, \ldots$, stopped at the first success, therefore succeed with probability at least $\frac{1}{4}$ in fewer than $8\sqrt{2}\,\kappa$ rounds (doubling_schedule_kappa).

### 5.4 Programs in the Paper's Regime

The circuits of this section become gate lists with the same semantics (qgates_sem) and run as programs. Their postcondition $|t\rangle\langle t|$ measures the fidelity with the target. For a program that initialises, runs a circuit with output $\psi$ and measures with $M$, where $|t\rangle$ lies in the range of $M_1$, we prove the triple $\models_t \{|\langle t|\psi\rangle|^2 I\}\ prog\ \{|t\rangle\langle t|\}$. For the repeat-until-success loop we prove the precondition $|\langle t|\psi\rangle|^2/\|M_1\psi\|^2$ (hoare_prog_overlap, hoare_rus_overlap). If the post-selected state is within $e$ of $|t\rangle$, the fidelity is at least $1-e^2$. With $e = 2(\varepsilon + 2(N-1)\eta)/(C\|A^{-1}b\|)$ from Theorem 6 and the oracle bound, one run satisfies $\{(Ct_0/\pi)^2(1-e^2)\,I\}$ and the loop satisfies $\{(1-e^2)\,I\}$ (hhl_apx_program, hhl_apx_rus). The filtered version gives $\{(1-\varepsilon^2)\,I\}$, and amplitude amplification as a program needs at most $\pi\kappa$ rounds (hhl_flag_rus, hhl_flag_amplified).

```
abbreviation (input) "apx_K ≡ N * (n * 2)"
abbreviation (input) "apx_O ≡ V ⊗ 1_m 2"
abbreviation (input) "apx_x ≡ herm_inv A *_v b"
abbreviation (input) "apx_t ≡ tensor_vec (win_state N) (tensor_vec
    (complex_of_real (1 / ‖apx_x‖) ·_v apx_x) (unit_vec 2 1))"
abbreviation (input) "apx_e ≡ 2 * (ε + 2 * real (N - 1) * η) / (C
    * ‖apx_x‖)"
theorem hhl_apx_rus:
  "state_sig.hoare_total_correct [N, n, 2] (complex_of_real (1 -
     apx_e^2) ·_m 1_m apx_K)
    (rus_meas apx_K (anc_proj apx_K) (qgates N (n * 2) apx_O
        (hhl_full_prog A t0 N C Prep))) (outer_prod apx_t apx_t)"
```

**Listing 13.** The repeat-until-success loop in the paper's regime (inside a context with the hypotheses of Theorem 6, an oracle V that is $\eta$-close to $e^{iAt_0}$ and a preparation Prep of the window state)

## 6 Down to Elementary Gates

The programs of Sections 4 and 5 use large gates: $H^{\otimes k}$, the controlled powers $\Lambda(U)$, $F_N$, the rotation stage, the preparations of $|b\rangle$ and of the clock, and the oracle $e^{iAt_0}$. A gate is *elementary* if it is a one- or two-qubit unitary acting on adjacent wires, $I_a \otimes G \otimes I_b$ with $G$ a $2 \times 2$ or $4 \times 4$ unitary (Listing 14). This section replaces every large gate by elementary ones. Each refinement proves that the product of the new gates equals the old gate, so every triple carries over (Section 2).

### 6.1 The Gate-Level Circuit

The Hadamard layer is $k$ single-qubit Hadamards. The controlled powers factor over the clock bits, $\Lambda(U) = \prod_j C_j(U^{2^j})$, where $C_j$ is controlled by bit $j$ of the clock (ctrl_pow_qubits). Equivalently, they use $2^k - 1$ singly controlled copies of $U$ (ctrl_pow_queries). The Fourier transform is the AFP's gate-level circuit, flattened into Hadamards, controlled phase rotations and swaps (qft_circuit). It has $(k^3 + 2k)/3$ gates, which is cubic because its controlled rotations are routed through nearest-neighbour swaps. The resulting circuit for HHL has $(2k^3 + 16k + 6)/3$ gates (length_hhl_elem_gates), and every gate is elementary except the oracle calls $C_j(U^{\pm 2^j})$, the preparation $U_b$ and the rotation stage (hhl_elem_gate_kinds).

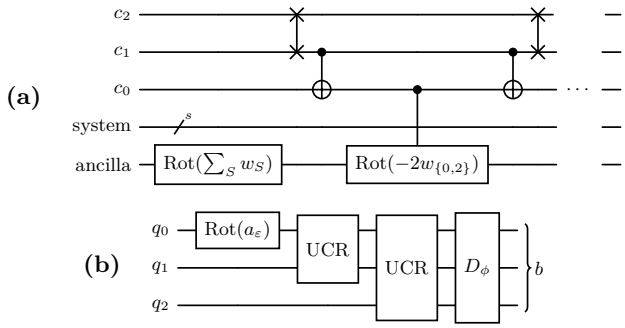



**Fig. 5.** (a) One block of the uniformly controlled rotation for $k = 3$ and $S = \{0, 2\}$: the SWAP and CNOT put $c_0 \oplus c_2$ into $c_0$, a rotation of the ancilla controlled by $c_0$ follows, and the chain is undone. The ancilla rotation controlled by $c_0$ crosses the system register, and is routed by adjacent swaps. (b) State preparation on three qubits. Level $j$ rotates qubit $j$ by an angle that depends on the qubits before it, $a_c = \arcsin(\|b_{c1}\|/\|b_c\|)$ for the blocks $b_c$ of $b$ indexed by the prefix $c$, as a uniformly controlled rotation; a diagonal unitary $D_\phi$ then sets the phases.

```
definition elementary :: "nat ⇒ complex mat ⇒ bool" where
  "elementary K U ⟷ (∃a b q G. q ∈ {1, 2} ∧ G ∈ carrier_mat (2 ^
      q) (2 ^ q) ∧ unitary G ∧ 0 < a ∧ 0 < b ∧
     K = a * (2 ^ q * b) ∧ U = 1_m a ⊗ (G ⊗ 1_m b))"
theorem qft_circuit:
  shows "circuit_mat (2 ^ n) (qft_circuit n) = fourier (2 ^ n)"
    and "U ∈ set (qft_circuit n) ⟹ elementary (2 ^ n) U"
theorem length_hhl_elem_gates: "3 * length (hhl_elem_gates A Ub t0
    k C) = 2 * k ^ 3 + 16 * k + 6"
```

**Listing 14.** Elementary gates, the Fourier circuit, and the gate count

### 6.2 The Rotation as a Uniformly Controlled Rotation

The rotation stage $\sum_c |c\rangle\langle c| \otimes I \otimes \mathrm{Rot}(\theta_c)$, with $\theta_c = \arcsin(C/\lambda_c)$ and $\mathrm{Rot}(a)$ the real rotation by $a$, applies a different angle for each of the $2^k$ clock values. Write the angles in the Walsh–Hadamard basis, $\theta_c = \sum_{S \subseteq \{0,\ldots,k-1\}} w_S (-1)^{\pi_S(c)}$, where $\pi_S(c)$ is the parity of the bits of $c$ in $S$ and $w_S = 2^{-k} \sum_{c'} \theta_{c'} (-1)^{\pi_S(c')}$ (walsh_inversion). Then the stage is one rotation of the ancilla by $\sum_S w_S$, followed by one block per non-empty $S$. A block computes $\pi_S(c)$ into the last clock qubit with a chain of nearest-neighbour CNOT and SWAP gates, rotates the ancilla by $-2w_S$ controlled by that qubit, and uncomputes the chain (Fig. 5a, ucr_circuit). For a system register of $s$ qubits, the controlled rotation is moved next to the clock by $s$ adjacent swaps (route_cv0).

```
theorem ucr_circuit:
  assumes n0: "0 < n"
  shows "circuit_mat (2 ^ k * (n * 2)) (ucr_gates k n θ) =
      ctrl_blocks (2 ^ k) (n * 2) (λc. 1_m n ⊗ Rot (θ c))"
theorem ucr_gate_kinds:
  assumes U: "U ∈ set (ucr_gates k n θ)" and n0: "0 < n"
  shows "elementary (2 ^ k * (n * 2)) U ∨ (∃a. U = cv0 k n (Rot
      a))"
```

**Listing 15.** The rotation stage as a uniformly controlled rotation

### 6.3 State Preparation

A state with non-negative amplitudes $a$ on $k$ qubits is prepared by $k$ levels of uniformly controlled rotations, the construction of Grover and Rudolph [12] and Möttönen et al. [21]. Level $j$ rotates qubit $j$ by the angle $\arcsin(\|a_{c1}\|/\|a_c\|)$ determined by the block norms (cascade_prepares, state_prep). The sine window is one instance (window_prep). For a complex $b$, the magnitudes are prepared first and the phases are set by a diagonal unitary. A diagonal unitary on $k+1$ qubits splits into a diagonal unitary on $k$ qubits, carrying the means of the phases of each pair, and a uniformly controlled rotation conjugated by a fixed one-qubit gate, carrying their differences (diag_gates_circuit). This yields the following theorem.

```
theorem bprep:
  assumes s: "1 ≤ s" and b: "dim_vec b = 2 ^ s" "‖b‖ = 1"
  shows "circuit_mat (2 ^ s) (bprep_gates s b) *_v unit_vec (2 ^ s)
      0 = b"
    and "⋀U. U ∈ set (bprep_gates s b) ⟹ elementary (2 ^ s) U"
    and "circuit_mat (2 ^ s) (bprep_gates s b) ∈ carrier_mat (2 ^
      s) (2 ^ s)"
    and "unitary (circuit_mat (2 ^ s) (bprep_gates s b))"
```

**Listing 16.** Every normalised $|b\rangle$ is prepared by elementary gates

### 6.4 Every Gate Elementary

For $n = 2^s$, the refinements combine into the program hhl_elementary: it satisfies the three triples of Theorems 4 and 5, and every gate is elementary or an oracle call $C_j(U^{\pm 2^j})$. The same holds for the paper-regime programs of Section 5.4 (hhl_apx_elementary), where the oracle is the approximate evolution $V$. The oracle calls themselves can be compiled. By the spectral theorem for normal matrices (taken from QHLProver), every unitary is $U = \prod_k (I + (\lambda_k - 1) p_k p_k^\dagger)$ for an orthonormal eigenbasis $(p_k)$. Each factor is $Q_k D_k Q_k^\dagger$, where $Q_k$ is the preparation circuit of $p_k$ and $D_k$ multiplies $|0\rangle$ by the phase $\lambda_k$. This gives exact universality, and hence HHL programs in which every gate is elementary.

```
theorem exact_universality:
  assumes U: "U ∈ carrier_mat (2 ^ m) (2 ^ m)" "unitary U" and m:
      "1 ≤ m"
  shows "∃Gs. (∀G ∈ set Gs. elementary (2 ^ m) G) ∧ circuit_mat (2
      ^ m) Gs = U"
theorem hhl_compiled:
  assumes ...
  defines "K ≡ 2 ^ k * (2 ^ s * 2)"
  shows "∃Gs. (∀G ∈ set Gs. elementary K G)
     ∧ state_sig.hoare_total_correct [2 ^ k, 2 ^ s, 2]
         (complex_of_real (C² * ‖herm_inv A *v b‖²) ·m 1m K)
           (hhl_prog K Gs) (anc_proj K 1)
     ∧ state_sig.hoare_total_correct [2 ^ k, 2 ^ s, 2] (1m K)
         (hhl_prog K Gs)
           (anc_proj K 0 + outer_prod (hhl_target A k b)
               (hhl_target A k b))
     ∧ (C ≠ 0 ⟶ state_sig.hoare_total_correct [2 ^ k, 2 ^ s, 2]
         (1m K) (hhl_rus_prog K Gs)
           (outer_prod (hhl_target A k b) (hhl_target A k b)))"
```

**Listing 17.** Exact universality, and HHL with every gate elementary (the triples are those of Listings 7 and 9; hypotheses as there, with $n = 2^s$ and $\|b\| = 1$)

Universality comes with no useful bound on the number of gates, so for cost statements the oracle calls remain the unit of measure. When $A$ is sparse, a better circuit for the oracle exists.

### 6.5 Sparse Hamiltonian Simulation

The paper assumes $s$-sparse $A$ and cites simulation results for $e^{iAt}$ [3]. We formalise the first-order version of this construction in three steps.

*Trotter.* For Hermitian $H_1, \ldots, H_m$ with $\|H_j\| \le L$,

$$\left\| (e^{itH_1/r} \cdots e^{itH_m/r})^r - e^{it(H_1+\cdots+H_m)} \right\| \le 2(mLt)^2/r$$

(trotter, Listing 18). The proof stays in the functional calculus. $|e^{ix} - 1 - ix| \le x^2$ bounds each factor to first order, and the errors add along products and powers of unitaries.

*Colouring.* The graph of an $s$-sparse Hermitian $A$ is edge-coloured greedily with $2s - 1$ colours. Each colour class is a matching, given by an involution $q_c$ of the index set, so $A$ is its diagonal plus $2s - 1$ Hermitian 1-sparse matrices whose entries are entries of $A$ (sparse_split) [2,3].

*The 1-sparse circuit.* Each 1-sparse evolution is implemented exactly on $|x\rangle|y\rangle|b\rangle$ (Fig. 6). A colour oracle $O_c$ writes the partner $q_c(x)$ into the second register. A comparator sorts the pair and records the order in $b$. An entry gate rotates $b$ by

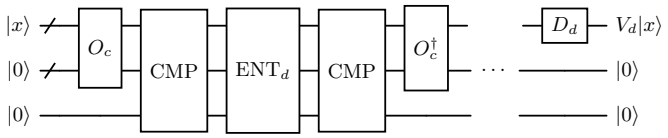



**Fig. 6.** One Trotter step for an $s$-sparse $A$ with $d = t/r$. For each of the $2s-1$ colours $c$, five gates implement $e^{idH_c}$ on $|x\rangle$ with both ancilla registers returned to $|0\rangle$, where $H_c$ is the 1-sparse part of colour $c$. A diagonal phase gate $D_d = e^{id\,\mathrm{diag}\,A}$ follows, so the step acts on $|x\rangle$ as the product $V_d$ of the $2s$ part evolutions. It has $5(2s-1)+1$ gates, and $r$ steps use $r(6s-2)$ oracle queries.

$e^{id\left(\begin{smallmatrix}0 & a\\ \bar{a} & 0\end{smallmatrix}\right)}$ with $a = A_{xy}$. The comparator and $O_c^\dagger$ then uncompute (colour_circ_basis). The oracle is a hypothesis, any unitary with $|x\rangle|0\rangle|b\rangle \mapsto |x\rangle|q_c(x)\rangle|b\rangle$, and one exists for every colouring. A final gate applies the phases of the diagonal.

```
theorem trotter:
  assumes Hs: "⋀H. H ∈ set Hs ⟹ herm_bounded n L H" and r: "0 < r"
  shows "op_close n (2 * (real (length Hs) * L * t)^2 / real r)
           (tprod n (t / real r) Hs ^_m r) (evolution (hsum n Hs)
               t)"
theorem sparse_split:
  assumes A: "A ∈ carrier_mat n n" "hermitian A" and sp: "sparse n
      s A" and s: "1 ≤ s"
  shows "∃p. edge_cover n (2 * s - 1) p (edges n A) ∧
           length (sparse_parts n A (2 * s - 1) p) = 2 * s ∧ hsum
               n (sparse_parts n A (2 * s - 1) p) = A"
theorem sparse_simulation_circuit:
  assumes A: "A ∈ carrier_mat n n" "hermitian A" and s: "1 ≤ s"
      and L: "0 ≤ L" and Lb: "⋀i j. i < n ⟹ j < n ⟹ cmod (A $$ (i,
          j)) ≤ L"
      and cov: "edge_cover n (2 * s - 1) p (edges n A)"
      and Os: "⋀c. c < 2 * s - 1 ⟹ colour_oracle n (p c) (Os c)"
          and r: "0 < r"
  shows "sg_run n (t / real r) A Os (sim_gates (2 * s - 1) r) *
      emb n = emb n * sim_mat n (t / real r) A (2 * s - 1) p r"
    and "sim_mat n (t / real r) A (2 * s - 1) p r ∈ carrier_mat n
        n"
    and "unitary (sim_mat n (t / real r) A (2 * s - 1) p r)"
    and "op_close n (8 * (real s * L * t)^2 / real r) (sim_mat n (t
        / real r) A (2 * s - 1) p r) (evolution A t)"
    and "length (filter is_query (sim_gates (2 * s - 1) r)) = r *
        (6 * s - 2)"
```

**Listing 18.** Trotter, the colouring, and the sparse simulation circuit

The circuit acts on inputs with both ancilla registers zero as a unitary $V$ on the system (emb embeds the system). $V$ is within $8(sLt)^2/r$ of $e^{itA}$, where $L$ bounds the entries of $A$. Plugged into the error bound of Section 5, this gives the output error $(\varepsilon + 16(N-1)(sLt_0)^2/r)\|b\|$ (hhl_sparse_window_error). The

**Table 2.** Errata in the HHL paper (arXiv:0811.3171v3). "Formal" names the theorem that refutes the claim or proves the correction.

| Where | Claim | Correction | Formal |
|---|---|---|---|
| Lemma 3 | $c = \pi^2/2$ | false for every $\kappa$ (at $\lambda = \frac{1}{2\kappa}$, $\bar{\lambda} = \frac{1}{\kappa}$); $c = \pi^2$ holds and is sharp | lemma3_paper_constant, lemma3, lemma3_sharp |
| Proof of Thm. 4 | $m = \frac{2}{\delta}\log(2n)/\log\log n$ rounds suffice | too few: for $n_0 = 2^{64}$, $\delta = \frac{1}{2}$ it gives $m = 130/3$, and after $m \le 44$ rounds the bound on $T_m$ still exceeds $n_0$; $m \ge \frac{2}{\delta}\ln(2n_0)$ suffices | paper_rounds_insufficient, corrected_rounds |
| (A34) | the dilation satisfies $\kappa^{-1}I \le A \le I$ | impossible: its diagonal is zero | herm_embed_diag |
| (A37) | the first qubit of $\boldsymbol{y}$ decides acceptance | the top half of $\boldsymbol{y}$ is always 0; permuting the dilation fixes it | first_half_perm, red_first_qubit |
| Thm. 6(2) | precision $\varepsilon = 2^{-n-2}$ | cannot separate counts $C$ and $C+1$ once the clock factor ($\le \frac{1}{4}$) is included | paper_precision_insufficient |
| App. A5 | $1/(3\varepsilon^2)$ samples | too few: $\varepsilon = \frac{1}{4}$, $p = \frac{1}{2}$ and 6 samples fail with probability $\frac{11}{16}$; $3/\varepsilon^2$ suffice | paper_sample_count_too_small, estimate_expectation |
| §IV | preconditioned system $(AB)c = Bb$ | neither left nor right preconditioning: in a $2 \times 2$ example its unique solution gives no solution of $Ax = b$ | paper_precondition_wrong |
| App. A5 | undo gates $U^\dagger_{3T+1-t}$ | must be $U^\dagger_{T+1-t}$ | construction red_U, red_inverse |
| App. A5 | $\kappa \le 2T$ | $\kappa = \coth\frac{1}{2T} > 2T$ when $3T$ is even (not formalised); we prove $\kappa \le 2T+1$ | red_A_bounds |

higher-order formulas behind the paper's cited cost, and the reversible arithmetic inside the oracles, are not formalised.

## 7 What the Formalisation Found

The HHL paper is a carefully written letter with a long appendix, and its main claims survive mechanisation. Its details do not all survive. Table 2 lists the statements of the arXiv version [14] that are false as written, with the correction and its formal witness. Most errors are in the appendix on the optimality of the algorithm, which reduces quantum circuits to matrix inversion (Theorems 4–6). Each optimality theorem is formalised up to the external complexity-theoretic fact it cites (PSPACE-completeness of TQBF, BQP $\subseteq$ PSPACE, and the classical lower bounds for Simon's problem and parity). The search lower bound of Theorem 4(2) is imported from the AFP entry *Compressed Oracles*.

The last two rows are corrections built into the formal construction rather than refutations: the reduction is defined with the correct undo gates, and only the weaker bound $2T + 1$ is proved. The other rows are refuted by explicit counterexamples. Three observations are worth recording. First, none of the errors we found invalidates a main claim of the paper: those are stated with $O(\cdot)$ and survive with other constants. Second, the errors sit where a proof sketch compresses a computation, for instance an evaluation of $f^2 + g^2$, an index, or a

number of rounds. Third, a recent re-derivation of the error analysis [17] remarks that the factor $\pi/2$ of Lemma 2 "is not correct for certain values of $\kappa$". With the paper's choice $\kappa' = 2\kappa$, which places the middle region at $[\frac{1}{2\kappa}, \frac{1}{\kappa})$, our proof hflag_lipschitz shows that $\lambda \mapsto |h(\lambda)\rangle$ is $\frac{\pi}{2}\kappa$-Lipschitz for every $\kappa > 0$, so Lemma 2 is correct as stated. A machine-checked proof settles such disagreements.

The lecture notes of Section 8 also contain false statements, all found because a statement failed to be provable as written. The Day 29 "Hamming $[7, 1, 3]$" code has distance 2 and cannot correct one error (notes_hamming_not_correcting). The Day 32 claim that distinct one-qubit Pauli errors map the Shor code to orthogonal subspaces fails for $Z_1 \neq Z_2$, which agree on the code space (shor_notes_Z1_Z2; the notes correct it on Day 33). A reformulation of the Day 21 conjecture is false for $n = 2$, $d = 1$ (notes_form_i_false). Two Day 23 examples, a formula of Day 31 and Cholesky's reduction on Day 5 each need an extra hypothesis.

## 8 The Foundation: Paulsen's Lectures, Formalised

The HHL development builds on a formalisation of one complete set of lecture notes: Vern Paulsen's *Lectures on Quantum Computing and Completely Positive Maps* (Houston, 2011) [22]. The notes are an unedited transcription of 34 lectures in five parts: basic mathematics, an introduction to quantum computing, completely positive maps, entanglement, and error correction. Taking a course as the specification forces one to formalise the whole supporting theory rather than the shortest path to a headline theorem. It also gives an external measure of completeness.

*Audit and document.* The repository's audit lists every definition, theorem, proposition, corollary, example and remark of the notes with its formal counterpart. Of the 170 items, 9 are deliberately not formalised (overviews, history, classical asides). Of the remaining 161, 156 are formalised faithfully, 5 in a variant form, and none is missing. The variants are a different proof route (Choi's Theorem 3 through a Radon–Nikodym theorem for completely positive maps, and Arveson's extension theorem by cone separation), or a statement of the notes that needs a hypothesis or is false (Section 7). The HHL paper is audited in the same way, in 23 items. An Isabelle-generated document of 404 pages restates the notes lecture by lecture, with the formal statements in place of the informal proofs, and it is rebuilt and re-checked by every build.

*Contents.* Lectures 1–10 give the linear algebra and the postulates: Schur's theorem, the spectral theorem, four characterisations of positivity, Cholesky's reduction, tensor products, measurements and POVMs. Part VI imports exactly this part. Lectures 11–22 give density matrices, partial traces, and completely positive maps with Choi's theorem (Listing 19) and the Choi–Kraus representation. Lectures 23–28 give the extreme points of the unital and of the trace-preserving channels (Choi's Theorem 3), the Hahn–Banach theorem on matrix

spaces, Arveson's extension theorem, and the separating-hyperplane theory of entanglement witnesses. Lectures 29–34 give classical and quantum codes, the Knill–Laflamme conditions in both directions, and Shor's nine-qubit code. One CPTP map corrects every single-qubit unitary error on each of the nine qubits (shor_code_theorem); the $2^9$-dimensional space is never handled entrywise but as three blocks of three qubits.

```
theorem choi_theorem:
  fixes Φ :: "complex mat ⇒ complex mat"
  assumes linear: "⋀A B. A ∈ carrier_mat n n ⟹ B ∈ carrier_mat n n
      ⟹ Φ (A + B) = Φ A + Φ B"
      and smult: "⋀c A. A ∈ carrier_mat n n ⟹ Φ (c ·m A) = c ·m Φ A"
      and range: "∀A. A ∈ carrier_mat n n ⟶ Φ A ∈ carrier_mat d d"
      and npos: "n > 0" and dpos: "d > 0"
  shows "completely_positive n d Φ ⟷ psd (choi_matrix n d Φ)"
```

**Listing 19.** Choi's theorem (Lecture 19)

*Two lessons.* First, HOL functions are total, while a map $M_n \to M_d$ is constrained only on the carrier. The naive definition of an extreme point ("$\Phi$ is not the midpoint of two different channels") is therefore vacuous, since any channel agrees on the carrier with maps that differ from it elsewhere. Extremality has to be defined through disagreement on the carrier. Second, two places in the notes genuinely need topology: the compactness of the unital channels, and the separation of a point from the closed convex cone of separable matrices. For these we wrap matrices of fixed dimension in a type, instantiate HOL-Analysis up to euclidean_space, and transport the statements back. The carrier-predicate world carries almost all of the development, and the typed world is used only where HOL-Analysis is applied.

*An open problem.* Lecture 21 asks whether every unital completely positive map $\Phi : M_n \to M_d$ is the average of $n$ unital maps of Choi rank at most $d$. This is the strong Ruskai–Audenaert conjecture for the dual channel, which is open in general [16]. We state it formally (D21_2) and prove the cases with direct constructions. If the Kraus operators split into $n$ groups that are orthogonal in a suitable sense, Fourier mixtures give the decomposition (fourier_split). This covers measurement maps and, via Horn's lemma, the case $d = 1$. Following Bacon, Smolin and Wolf, every unital map with $n = 2$ is such an average (ra_two): the off-diagonal block of the Choi matrix is $R_1 K R_2^\dagger$ for a contraction $K$, and $K = \frac{1}{2}(U_1 + U_2)$ for unitaries obtained from a singular value decomposition, which we build from the spectral theorem. A reformulation in the notes turned out to be false (Section 7).

*Size.* The formalisation of the notes has 50 theories with 51 729 lines and 1 614 results (lemmas, theorems, corollaries and propositions). Part VI has 29 theories with 22 174 lines and 1 161 results, plus an 80-line bridge to the AFP's quantum search bound in a separate session (Table 3).

## 9 Related Work

*Verified HHL.* Zhou, Yu and Ying introduced an applied quantum Hoare logic with projection predicates and rules for robustness, and used it for a pen-and-paper verification of HHL [34]. CoqQ [33] formalised that proof in Coq. Under the assumption that every $\lambda_j t_0/2\pi$ lies in $\{1, \ldots, N-1\}$, it proves $\models_p \{I\}$ *HHL* $\{|x\rangle\langle x|\}$ for the repeat-until-success program, in about 280 lines. $U_b$ and the controlled rotation are built from partial specifications, and the controlled evolution is a built-in multiplexer. Wu, Deng and Xu [30] verify the same exact HHL, and Shor's algorithm, in a quantum Hoare logic with distribution formulas embedded in Coq. Our development is in Isabelle/HOL and differs in scope. It proves total correctness, including termination and the distribution of the number of rounds; it treats the regime of the paper's error analysis, with inexact eigenvalues, an approximate oracle, the filter functions and amplitude amplification; it compiles the programs to elementary gates and simulates sparse Hamiltonians; and it records the errata of Section 7.

*Other verified quantum algorithms.* In Coq, SQIR and its applications verify phase estimation, Grover's algorithm and others [15], and an end-to-end implementation of Shor's algorithm, including the reversible arithmetic of modular exponentiation [23]. In Why3, QBricks verifies phase estimation, Grover's algorithm and the order-finding part of Shor's algorithm [9]. A recent HOL Light development treats Grover's algorithm [24]. In Isabelle, Isabelle Marries Dirac formalises qubits, gates, teleportation, the no-cloning theorem and the Deutsch–Jozsa algorithm [6,5]. QHLProver verifies Grover's algorithm with the logic we use [18], and the AFP contains the quantum Fourier transform [20]. Unruh's libraries develop bounded operators, tensor products of Hilbert spaces and Kraus maps in infinite dimension [8,27,29]; they underlie his quantum relational Hoare logic [26] and the compressed-oracle technique, whose search lower bound [28] we import for Theorem 4(2) of the HHL paper. We also import Sylvester's criterion from the AFP [4].

*Error analyses of HHL.* Li and Phillips [17] re-derive the paper's error analysis by hand, with corrections of typos and with more detail on the amplitudes after phase estimation. Their remark on Lemma 2 is discussed in Section 7. The later literature improves the dependence on $\kappa$ and $\varepsilon$ [10]. Verifying those algorithms is future work.

## 10 Discussion

*What is trusted.* Beyond Isabelle's kernel, the statements depend on the definitions they use. Those of the AFP entries Jordan-Normal-Form, Isabelle Marries Dirac and QHLProver fix matrices, gates, programs, their semantics and the meaning of triples. Our own definitions are the ones shown in the listings: the grid values reg_eig, the functional calculus, the eight gates and the programs,

**Table 3.** Size of the development (results are lemmas, theorems, corollaries and propositions).

| Part | Theories | Lines | Results |
|---|---|---|---|
| Mathematics (functional calculus, phase estimation) | 4 | 1 133 | 73 |
| Algorithm (circuits, error analysis, rest of the paper) | 13 | 11 371 | 561 |
| Program (QHLProver programs and triples) | 5 | 3 593 | 187 |
| Circuit (elementary gates, sparse simulation) | 7 | 6 077 | 340 |
| Part VI, HHL | 29 | 22 174 | 1 161 |
| Bridge to the AFP search bound (separate session) | 1 | 80 | 4 |
| Paulsen's lectures (the foundation) | 50 | 51 729 | 1 614 |

the filter functions, the predicate elementary, and the colour oracle. A reader who accepts the listings and the AFP's semantics can check that the theorems say what this paper claims.

*Assumptions and limitations.* The preparation of $|b\rangle$ is a hypothesis ($U_b|0\rangle = b$) in Sections 4–5 and a circuit in Section 6. The colour oracles of the sparse simulation are hypotheses about their action on basis states. The reversible arithmetic that would implement them, and the higher-order product formulas behind the paper's cost, are not formalised. Universality is exact but gives no gate count, and the routing of the controlled rotation assumes $n = 2^s$. Costs are stated as numbers of gates, oracle calls and rounds, not as membership in complexity classes. The optimality theorems are formalised up to the complexity-theoretic facts they cite.

*Lessons.* Four proof patterns carried most of the work. First, statements about every right-hand side are proved for eigenvectors and extended by the lemma that matrices agreeing on all eigenvectors are equal, so no sum over an eigenbasis appears. Second, register sums $\sum_j |j\rangle \otimes f_j$ make block-diagonal operators act branch by branch, which turns the error analysis into scalar estimates. Third, every triple is proved for all gate lists with a given product, so each refinement towards elementary gates is a circuit identity with no new program reasoning. Fourth, the total correctness of repeat-until-success loops reduces to a geometric series once the body resets the state. On the side of the tools, an oracle audit (Thm_Deps.all_oracles) is meaningful only for builds with `quick_and_dirty = false`. With `quick_and_dirty = true`, Isabelle discharges internal goals of the fun and record packages by Skip_Proof.cheat_tac, and every theorem downstream of such a definition reports the oracle skip_proof, even in a development without a single `sorry`.

## 11 Conclusion

We have verified the HHL algorithm in Isabelle/HOL as programs of a quantum while-language, with quantum Hoare logic as the common language from

the functional calculus up to elementary gates. In the exact case the program is totally correct: it succeeds with probability $C^2\|A^{-1}b\|^2$, it is exact on success, and its repeat-until-success loop terminates with probability one after $1/p$ expected rounds. In the paper's regime, the bounds of Theorem 1 and the run time hold with explicit constants, and the programs satisfy the corresponding fidelity triples. All of this persists down to circuits of one- and two-qubit gates, including the preparation of $|b\rangle$ and, for sparse $A$, the simulation of $e^{iAt}$. On the way, the formalisation corrected the constant of Lemma 3 and several statements of the paper's optimality proofs.

The development is also an argument for formalising a whole course before an algorithm. Paulsen's notes supplied the spectral theorem and the positivity theory that HHL needs. Their later lectures supply the completely positive maps and Choi–Kraus representations that are the semantics of quantum programs, which suggests the next step: proving that QHLProver's denotations are the channels of Lecture 19, so that the two halves of the development meet in one theorem. Higher-order simulation with verified arithmetic oracles, gate-count bounds for compilation, and the later linear-systems algorithms are further natural continuations.

*Acknowledgements.* The lecture notes were written by Vern Paulsen and transcribed by Brandon Lee. The formalisation was carried out with the assistance of AI coding agents, and all statements were checked by Isabelle.

## References

1. Aaronson, S.: Read the fine print. Nature Physics **11**(4), 291–293 (2015)
2. Aharonov, D., Ta-Shma, A.: Adiabatic quantum state generation and statistical zero knowledge. In: 35th Annual ACM Symposium on Theory of Computing (STOC 2003). pp. 20–29. ACM (2003)
3. Berry, D.W., Ahokas, G., Cleve, R., Sanders, B.C.: Efficient quantum algorithms for simulating sparse Hamiltonians. Communications in Mathematical Physics **270**(2), 359–371 (2007)
4. Binder, S., Jiang, Z.: Two theorems on Hermitian matrices. Archive of Formal Proofs, `https://isa-afp.org/entries/Two_Hermitian_Results.html` (Nov 2024)
5. Bordg, A., Lachnitt, H., He, Y.: Isabelle marries Dirac: a library for quantum computation and quantum information. Archive of Formal Proofs, `https://isa-afp.org/entries/Isabelle_Marries_Dirac.html` (Nov 2020)
6. Bordg, A., Lachnitt, H., He, Y.: Certified quantum computation in Isabelle/HOL. Journal of Automated Reasoning **65**, 691–709 (2021). `https://doi.org/10.1007/s10817-020-09584-7`
7. Brassard, G., Høyer, P., Mosca, M., Tapp, A.: Quantum amplitude amplification and estimation. In: Quantum Computation and Information, Contemporary Mathematics, vol. 305, pp. 53–74. American Mathematical Society (2002)
8. Caballero, J.M.R., Unruh, D.: Complex bounded operators. Archive of Formal Proofs, `https://isa-afp.org/entries/Complex_Bounded_Operators.html` (Sep 2021)

9. Chareton, C., Bardin, S., Bobot, F., Perrelle, V., Valiron, B.: An automated deductive verification framework for circuit-building quantum programs. In: Programming Languages and Systems (ESOP 2021). LNCS, vol. 12648, pp. 148–177. Springer (2021)
10. Childs, A.M., Kothari, R., Somma, R.D.: Quantum algorithm for systems of linear equations with exponentially improved dependence on precision. SIAM Journal on Computing **46**(6), 1920–1950 (2017)
11. D'Hondt, E., Panangaden, P.: Quantum weakest preconditions. Mathematical Structures in Computer Science **16**(3), 429–451 (2006)
12. Grover, L., Rudolph, T.: Creating superpositions that correspond to efficiently integrable probability distributions. arXiv:quant-ph/0208112 (2002)
13. Harrow, A.W., Hassidim, A., Lloyd, S.: Quantum algorithm for linear systems of equations. Physical Review Letters **103**, 150502 (2009)
14. Harrow, A.W., Hassidim, A., Lloyd, S.: Quantum algorithm for solving linear systems of equations. arXiv:0811.3171v3 (2009)
15. Hietala, K., Rand, R., Hung, S.H., Li, L., Hicks, M.: Proving quantum programs correct. In: Interactive Theorem Proving (ITP 2021). LIPIcs, vol. 193, pp. 21:1–21:19 (2021)
16. Kumar, N., Wolf, M.M.: The Ruskai–Audenaert conjecture & equipartitions of positive operators. arXiv:2607.23066 (2026)
17. Li, X., Phillips, C.: Detailed error analysis of the HHL algorithm. arXiv:2401.17182 (2024)
18. Liu, J., Zhan, B., Wang, S., Ying, S., Liu, T., Li, Y., Ying, M., Zhan, N.: Formal verification of quantum algorithms using quantum Hoare logic. In: Computer Aided Verification (CAV 2019). LNCS, vol. 11562, pp. 187–207. Springer (2019). https://doi.org/10.1007/978-3-030-25543-5_12
19. Liu, J., Zhan, B., Wang, S., Ying, S., Liu, T., Li, Y., Ying, M., Zhan, N.: Quantum Hoare logic. Archive of Formal Proofs, https://isa-afp.org/entries/QHLProver.html (Mar 2019)
20. Manrique, P.: Quantum Fourier transform. Archive of Formal Proofs, https://isa-afp.org/entries/Quantum_Fourier_Transform.html (Jan 2025)
21. Möttönen, M., Vartiainen, J.J., Bergholm, V., Salomaa, M.M.: Transformation of quantum states using uniformly controlled rotations. Quantum Information and Computation **5**(6), 467–473 (2005)
22. Paulsen, V.: Lectures on quantum computing and completely positive maps (2011), lecture notes, University of Houston, Fall 2011; transcribed by Brandon Lee, December 16, 2011. Unpublished
23. Peng, Y., Hietala, K., Tao, R., Li, L., Rand, R., Hicks, M., Wu, X.: A formally certified end-to-end implementation of Shor's factorization algorithm. Proceedings of the National Academy of Sciences **120**(21), e2218775120 (2023)
24. Sun, H., Shi, Z., Chen, S., Wang, G., Li, X., Guan, Y., Zhang, Q., Shao, Z.: Formal modeling and verification of Grover's algorithm. arXiv:2601.02435 (2026)
25. Thiemann, R., Yamada, A.: Matrices, Jordan normal forms, and spectral radius theory. Archive of Formal Proofs, https://isa-afp.org/entries/Jordan_Normal_Form.html (Aug 2015)
26. Unruh, D.: Quantum relational Hoare logic. Proceedings of the ACM on Programming Languages **3**(POPL), 33:1–33:31 (2019)
27. Unruh, D.: The tensor product on Hilbert spaces. Archive of Formal Proofs, https://isa-afp.org/entries/Hilbert_Space_Tensor_Product.html (Sep 2024)
28. Unruh, D.: Compressed oracles. Archive of Formal Proofs, https://isa-afp.org/entries/Compressed_Oracles.html (Dec 2025)

29. Unruh, D.: Kraus maps. Archive of Formal Proofs, https://isa-afp.org/entries/Kraus_Maps.html (Jun 2025)
30. Wu, H., Deng, Y., Xu, M.: Local reasoning about probabilistic behaviour for classical–quantum programs. arXiv:2308.04741 (2023), VMCAI 2024
31. Ying, M.: Floyd–Hoare logic for quantum programs. ACM Transactions on Programming Languages and Systems **33**(6), 19:1–19:49 (2011)
32. Ying, M.: Foundations of Quantum Programming. Morgan Kaufmann (2016)
33. Zhou, L., Barthe, G., Strub, P.Y., Liu, J., Ying, M.: CoqQ: Foundational verification of quantum programs. Proceedings of the ACM on Programming Languages **7**(POPL), 833–865 (2023). https://doi.org/10.1145/3571222
34. Zhou, L., Yu, N., Ying, M.: An applied quantum Hoare logic. In: Programming Language Design and Implementation (PLDI 2019). pp. 1149–1162. ACM (2019). https://doi.org/10.1145/3314221.3314584

# A The Theories of Part VI

**Table 4.** The theories of Part VI (session Vern_Paulsen_QC_Algorithms), by level, with their lines and results. The bridge HHL_Search_Bound (80 lines, 4 results) is a separate session because it imports the AFP entry *Compressed Oracles*.

| Theory | Content | Lines | Results |
|---|---|---|---|
| *Mathematics* | | | |
| Hermitian_Functional_Calculus | $g(A)$, $e^{itA}$, $A^{-1}$ | 423 | 36 |
| Phase_Estimation | Fourier matrix, phase estimation, $4/\pi^2$ | 502 | 26 |
| QFT_Fourier | the AFP QFT circuit is $F_{2^n}$ | 104 | 4 |
| QPE_Register | $H^{\otimes n}\|0\cdots0\rangle$ is uniform | 104 | 7 |
| *Algorithm* | | | |
| HHL | exact circuit, solution, $1/\kappa^2$, dilation | 816 | 46 |
| HHL_Trajectory | the eight gates and their states | 591 | 28 |
| HHL_Approximate | inexact phase estimation, clamping | 1 174 | 53 |
| HHL_Window | sine window, $N \sim 1/\varepsilon$ | 1 210 | 46 |
| HHL_Simulation | oracle circuits, robustness | 745 | 49 |
| HHL_Complete | readout, amplitude amplification | 865 | 42 |
| HHL_Filter | filter functions, Lemmas 2–3 | 1 548 | 68 |
| HHL_Filter_Bounds | Theorem 1 for every $b$ | 816 | 29 |
| HHL_Runtime | (A32), BHMT averaging, run time | 611 | 21 |
| HHL_Reduction | circuits to matrix inversion | 647 | 55 |
| HHL_Rectangular | rectangular $A$, pseudo-inverse | 385 | 20 |
| HHL_Discussion | $f(A)\|b\rangle$, SWAP test, estimation | 908 | 44 |
| HHL_Optimality | Theorems 4–6 up to cited facts | 1 055 | 60 |
| *Program* | | | |
| HHL_Program | initialisation, programs, triples | 1 402 | 89 |
| HHL_Termination | repeat until success: total correctness | 502 | 19 |
| HHL_Paper_Program | the paper's regime as programs | 877 | 31 |
| HHL_Gate_Refinements | rotation split, window reflection | 635 | 29 |
| HHL_Preconditioned | preconditioned systems | 177 | 19 |
| *Circuit* | | | |
| HHL_Gates | elementary gates, QFT circuit, count | 895 | 75 |
| HHL_Uniform_Rotation | uniformly controlled rotation | 1 142 | 73 |
| HHL_State_Preparation | state preparation, window | 785 | 32 |
| HHL_All_Elementary | $\|b\rangle$, routing, exact universality | 1 031 | 44 |
| HHL_Trotter | Lie–Trotter with explicit error | 572 | 26 |
| HHL_Sparse_Split | sparse $A$ as $2s$ 1-sparse parts | 460 | 20 |
| HHL_Sparse_Circuit | the sparse simulation circuit | 1 192 | 70 |